AXON-RFC-006 Appendix C (v0.1): OpenAI Compatibility as Agent Dispatch View, Capability-URI API Keys

LLM endpoints are not a new ontology.
Every `<agent>.chat` ability is an OpenAI-shape
`/v1/chat/completions` model after one stream filter.

Silan Hu
National University of Singapore

Draft v0.1, May 4, 2026

Abstract

RFC-006-B v0.6 established the discipline that external protocols enter EasyNet as transport views over the invocation graph rather than as new ontologies. Pages was the first reference instance: HTTP file fetches become deterministic projections of the `<user>.<project>.page.fetch` ability. This appendix specifies the second reference instance: an OpenAI-compatible `/v1/chat/completions` endpoint exposed by the Hub on top of any agent's chat-base ability output. The construction is not a wrap. The dispatched agent — Claude Code, Codex, or any future driver — already emits a streaming sequence of typed frames; those frames, after a small deterministic filter, are exactly the wire shape of an OpenAI streaming completion. The adapter is a view. Authentication is not OAuth: API keys are minted by keyring abilities and addressed as `resource/api_key.<id>` URAs; presence of the token at the boundary is presence of a capability, and revocation is a single keyring call. Four normative invariants centre v0.1: Adapter Purity (the OpenAI surface mutates nothing it would not have mutated through the regular dispatch path), Capability-URI Key (API keys are URAs, not OAuth tokens), Filter Determinism (the stream-json → SSE projection is deterministic over the dispatch sequence), and Auth Receipt Trail (every request mints one canonical receipt against the API-key resource, all subsequent dispatch receipts cite it). The MVP is two endpoints on the Hub — `/v1/chat/completions` and `/v1/models` — backed by a fixed-shape stream filter implemented in the Hub agent's `openai.chat_completions` ability. Cursor, Continue, openai-python, and any other OpenAI-shape client work against EasyNet without code changes.

Contents

1 Position in the RFC-006 series

[derived] RFC-006-B v0.6 made one paradigm explicit:

> External protocols are transport views over the invocation graph, not new ontologies.

It then realised the paradigm against one external protocol — HTTP file fetch — and called the realisation *Pages*. This appendix realises the same paradigm against a second external protocol — the OpenAI `/v1/chat/completions` streaming wire format — and calls the realisation *OpenAI compat*. The structure of the appendix mirrors RFC-006-B section by section, so a reader can read the two side by side and verify that the same disciplines apply.

Aspect	RFC-006-B (Pages)	RFC-006-C (this)
External protocol	HTTP / GET / static files	OpenAI `/v1/chat/completions` (streaming)
Underlying ability	`<user>.<project>.page.fetch`	`<agent>.chat` (or any chat-base ability)
Hub adapter ability	`01HUB.pages.serve`	`01HUB.openai.chat_completions`
URL form	`<project>.<user>.pages.<realm>.<hash>/`	`<agent>/v1/chat/completions`
Is the adapter pure?	Yes (RFC-006-B INV-1)	Yes (this RFC INV-1)
Identity addressing	`resource/<user>.<project>/<path>`	`resource/api_key.<id>`
Determinism guarantee	Same id → same bytes (RFC-006-B INV-3)	Same dispatch frames → same SSE (this RFC INV-3)
What ships in the MVP	two files (hello-world.html + style.css)	one streaming completion against `web-builder.chat`

The two appendices share the same vocabulary, the same kind of invariant numbers, the same closing claim. RFC-006-D, RFC-006-E, RFC-006-F (file storage, conversation snapshots, mission outputs) will follow the same structure.

2 The paradigm, restated

2.1 Statement

[normative]**An external streaming protocol is a transport view over an agent's dispatch sequence.** The wire shape an external client receives is a deterministic projection of the typed frames the agent emits during dispatch. The adapter that produces the projection runs as an ability rooted at the Hub, mints one envelope per request, and emits the projection as its body.

The agent dispatch sequence is the source of truth. The OpenAI SSE stream is one of its views. Other views may co-exist (Anthropic `messages` streaming, Gemini `generateContent`, future protocols) without changing the source-of-truth side. Each view is a pure filter: a different filter, a different projection, the same dispatch.

2.2 Why this is not a chat-API wrapper

[derived] A naive design would introduce a parallel `chat_completions` primitive in EasyNet — a new state_type, a new receipt class, a new ability namespace — and forward to it. This is what *wrapping* looks like, and it is structurally wrong here for the same reason RFC-006-B v0.6 rejected "new state_type for web_app": it duplicates an ontology that already exists. EasyNet already has a notion of "the streaming output of an agent's work" — the dispatch sequence the chat ability is built on. The OpenAI surface projects that sequence; it does not host an alternative one.

2.3 Why a chat-base ability is already a streaming completion

[derived] Every agent driver currently registered with EasyNet (Claude Code via stream-json, Codex via SSE) emits a typed-frame stream during dispatch. The frame types and an OpenAI streaming completion's chunk types map onto each other:

Agent dispatch frame	OpenAI SSE chunk
`system/init` (driver handshake)	dropped
`assistant.message.text` (incremental tokens)	`choices[].delta.content`
`assistant.message.tool_use`	`choices[].delta.tool_calls[]` (when client requested tool surface; otherwise dropped or summarised)
`user.tool_result`	dropped (internal dispatch loop)
`assistant.message.thinking`	optional; mapped to `choices[].delta.reasoning` when the client opts in
`result` (terminal)	`choices[].finish_reason` + `[DONE]` sentinel

The mapping is deterministic, line-for-line, with no state-bearing transformations. INV-3 makes this normative.

2.4 The minimum reference instance: web-builder.chat

[derived] The smallest instance with all four ingredients (an external protocol, a deterministic streaming filter, a capability-style auth, a complete receipt trail) is exposing the existing `web-builder.chat` ability as `model = "web-builder"` on a single `/v1/chat/completions` endpoint. Cursor / Continue / openai-python configured against the EasyNet Hub URL with an EasyNet-issued API key call `web-builder` the same way they would call `gpt-4o`. The first end-to-end demo is one Cursor request that lands inside web-builder's dispatch, picks up the agent's real reply, and returns it byte-equivalent to the wire shape Cursor expects.

3 Assumptions

[normative] v0.1 is written under the following assumptions. Each is stated explicitly so downstream implementation work can be discharged independently while this appendix's semantic claims remain stable.

Assumption — A-1: Chat-base abilities exist

EasyNet has at least one ability of the form `<agent>.chat` whose dispatch produces a streaming sequence of typed frames (text, tool_use, tool_result, result). The current implementation registers `<agent>.chat` for each registered agent through `chat_ability::register` (RFC-001 Phase 3 et al.). v0.1 does not change that surface.

Assumption — A-2: Keyring abilities mint resource URAs

The keyring family (RFC-002 Phase 3, vault-style) supports ability invocations that mint long-lived capability tokens addressed as `resource/api_key.<id>` URAs. Whether the mint primitive is named `<user>.keyring.mint_api_key`, `<user>.api_key.create`, or something else is implementation detail; what matters is that:

- the minted URA is conformant to v4.1.5 (three URA segments, `resource/api_key.<id>`);
- presence of `<id>` at the HTTP boundary cryptographically authenticates the bearer (RFC-002 §3 vault rules apply);
- revocation is a single ability call.

Assumption — A-3: Receipts can be chained cross-class

The receipt store can record both canonical (state-changing) and operational (state-preserving) receipts and link them through `causal_context.Scalar`. INV-4's receipt trail relies on this; it is the same machinery RFC-006-B v0.6 §5 uses for receipt-chain audit walking.

Assumption — A-4: The Hub agent owns its adapter abilities

`01HUB.openai.chat_completions` is a hub-rooted ability in the same sense `01HUB.pages.serve` is. The *ability ownership widens to (user, agent, hub, resource)* amendment from RFC-006-B Phase 0 covers this case verbatim.

4 The four normative invariants

4.1 INV-1 — Adapter Purity

Invariant — INV-1: Adapter Purity (OpenAI)

`ability/01HUB.openai.chat_completions`, the ability that bridges the OpenAI streaming protocol into the invocation graph, **MUST** be a pure transport adapter. Specifically:

- It **MUST NOT** mutate any agent state.
- It **MUST** emit *exactly one* canonical receipt per OpenAI request, against the API-key resource (INV-4); no other canonical receipts originate from this adapter.
- It **MAY** emit operational receipts (per-frame or aggregated; lossy observability).
- It **MUST** delegate the dispatch entirely to the underlying chat-base ability via the standard `forward_invoke` dispatch path. It **MUST NOT** embed an alternative dispatcher.
- It **MUST NOT** cache, summarise, or transform the agent's reply beyond mechanical wire-shape framing (chunking by SSE `data:` lines; closing with `[DONE]`).

[derived] Adapter Purity rules out the entire failure mode in which the OpenAI surface accidentally becomes a parallel authoritative state plane: a Hub implementation that adds "output filtering" or "response rewriting" beyond the wire-shape transformation has stopped being an adapter and is now an authority. INV-1 forbids it at the specification layer, not at the implementation layer.

4.2 INV-2 — Capability-URI Key

Invariant — INV-2: API Key as Capability Resource

Every API key issued for OpenAI compatibility **MUST** correspond to a v4.1.5-conformant URA of the form `resource/api_key.<id>`, where `<id>` is an unguessable token of at least 256 bits of entropy. The URA **MUST**:

- be minted by a keyring ability (A-2);
- resolve, via the keyring, to a *single* `user/<username>` URA — the user the key represents at admission time;
- expose at least one revocation ability so the key can be deactivated without restarting the daemon;
- carry zero conversation state of its own (the key is the capability; it does not hold messages, threads, or cached context).

The HTTP `Authorization: Bearer <id>` header is the transport rendering of the URA. Presence of the header is *the* authentication signal — no OAuth, no token exchange, no scope object.

Note

The minimal implementation gives each key a flat "acts-as `user/<username>`" grant. Future revisions may attach narrower scopes (e.g., "can call only models matching `<owner>..chat`") by recording them on the key resource at mint time. Scope refinement is additive; v0.1 ships flat keys.

4.3 INV-3 — Filter Determinism

Invariant — INV-3: Stream Filter Determinism

The function from agent dispatch frame sequence to OpenAI SSE chunk sequence **MUST** be a pure deterministic projection. For a given dispatch sequence D and a given filter configuration F:

$$\texttt{sse}(D, F) = \texttt{filter}(D, F)$$

where `filter` contains no time, no random, no host- specific data, no agent identity beyond what is already in D, and no model-output post-processing.
It follows that:

- Two Hub implementations on different hosts running the same filter configuration against the same dispatch sequence **MUST** produce byte-identical SSE streams (modulo HTTP-layer fragmentation that does not change the bytes after re-assembly).
- A receipt-chain auditor walking from the canonical auth receipt down through dispatch receipts **MUST** be able to recompute the SSE bytes the client received, given the dispatch frames the receipts cite.
- The filter **MUST NOT** introduce dynamic content (timestamps, IP-derived data, server-name banners). Any such content belongs in the dispatch sequence itself, not in the projection.

[derived] INV-3 is what makes OpenAI compat federate. Without it, two Hubs claiming the same API key resource may serve different bytes for the same request; with it, a CDN, a logging proxy, or an auditor can rely on the dispatch sequence as the canonical record and treat the SSE bytes as a function of it.

Note

INV-3 does not forbid all dynamic content in chat replies; it forbids dynamic content from *the filter*. The agent's own dispatch may legitimately include timestamps the LLM produced ("it is now 2026-05-04"). What INV-3 forbids is the adapter adding such content unilaterally.

4.4 INV-4 — Auth Receipt Trail

Invariant — INV-4: Auth Receipt Trail

Every OpenAI request received at `/v1/chat/completions` **MUST**:

1. Mint one canonical receipt with:
   - `ability_uri = ability/01HUB.openai.chat_completions`
   - `caller_uri = hub/01HUB`
   - `callee_uri = hub/01HUB`
   - `subject_uri = resource/api_key.<id>`
   - `state_transition.body_hash = sha256(canonicalised request body)`
   - `state_transition.usage = { prompt_tokens, completion_tokens, total_tokens, model }`
2. Forward dispatch through the resolved chat-base ability with `causal_context.Scalar = <id of canonical receipt above>` so each dispatch-side receipt is chain-walkable back to the auth receipt.
3. Honour an `api_key` revocation that lands after admission but before dispatch terminates by surfacing a structured stream-end event (the OpenAI client receives a `[DONE]` after a final `finish_reason = "cancelled"` chunk).

[derived] INV-4 is the adapter's narrow but real authoritative-state contribution: "a request was authorised under this key at this moment with this body hash and these usage numbers." Everything else stays in dispatch land.

4.5 Why these four, no fewer, no more

[derived] Following the degeneracy method of AXIOM.tex §why-six and RFC-006-B v0.6 §invariants:

- **Without INV-1**, the OpenAI surface is a second authoritative state plane. An operator with access to the Hub process can rewrite chat replies without going through the dispatch path; the receipt chain is no longer the single source of truth.
- **Without INV-2**, API keys carry their own semantics outside the URA system. Revocation, scoping, auditing all become bespoke. Federation across realms becomes impossible because a key issued by one realm has no canonical name in another.
- **Without INV-3**, the same model on two hosts returns different bytes for the same request. CDN caching, third-party audit, and replay verification all collapse.
- **Without INV-4**, an OpenAI request enters EasyNet without a primary ledger entry. The receipt chain has gaps: "the agent dispatched, but on whose authority?"

Each invariant blocks a class of failure that the others do not address; removing any one collapses the model.

5 State transition model in detail

5.1 Mint key: `<user>.keyring.mint_api_key`

[normative] The user-rooted keyring ability that mints an OpenAI API key for that user. Canonical, state-changing.

```
caller   = caller_agent_uri
callee   = caller_agent_uri                    (self-call)
ability  = ability/<user>.keyring.mint_api_key
subject  = user/<user>                          (the user the key acts as)
args     = {
    label:       "cursor on alice's mac",      (optional human label)
    expires_at:  null | "<rfc3339>"            (null = no auto-expiry)
}
```

Pre-state: zero or more `api_key` resources owned by the user.

Post-state: one new `resource/api_key.<id>` resource exists, owned by the user, status = `active`.

Receipt class: canonical. The receipt's `state_transition.post_state` carries the minted URA; `state_transition.token_hash` carries sha256(token_bytes) so the chain proves a specific bearer existed without storing the secret.

5.2 Authorise: `01HUB.openai.chat_completions`

[normative] The Hub-rooted adapter ability invoked by the HTTP boundary on every request. Canonical (one auth receipt per call) but adapter-pure (no other state mutation).

```
caller   = hub/01HUB                           (Hub-as-caller)
callee   = hub/01HUB                           (self-call)
ability  = ability/01HUB.openai.chat_completions
subject  = resource/api_key.<id>               (parsed from Authorization)
args     = {
    request: {
        model:    "<ability URA tail>",        (e.g. "web-builder")
        messages: [...],
        stream:   true,
        ...
    }
}
```

Pre-state: `resource/api_key.<id>` exists with status = `active`.

Post-state: same. No state mutation in the adapter.

Receipt class: canonical (per INV-4). The receipt records body hash and usage; subsequent dispatch receipts cite this receipt as `causal_context.Scalar`.

5.3 Dispatch: `<agent>.chat`

[normative] The chat-base ability the OpenAI request resolved to. Operational, state-preserving (the agent's reply is a function of input + agent state at this instant; no canonical state is mutated by reading the reply).

```
caller   = hub/01HUB
callee   = agent/<resolved-from-model-name>
```

```
ability  = ability/<agent>.chat
subject  = user/<resolved-from-api-key>
args     = { prompt: <last user message>, context: <other messages> }
nonce    = <derived from the canonical auth receipt id>
causal_context = Scalar(<canonical auth receipt id>)
```

The dispatch's own typed-frame stream is what the filter projects to OpenAI SSE.

Receipt class: operational; one per call, with frame-level operational sub-receipts as the dispatch produces tool calls / sub-invocations.

5.4 Revoke: `<user>.keyring.revoke`

[normative] Revocation primitive. Canonical, state-changing.

```
caller   = caller_agent_uri
callee   = caller_agent_uri
ability  = ability/<user>.keyring.revoke
subject  = resource/api_key.<id>
args     = { reason: "user-initiated" | "compromised" | ... }
```

Pre-state: `api_key` resource exists with status = `active`.

Post-state: same resource exists with status = `revoked`, `revoked_at` populated. Subsequent OpenAI requests with that bearer are rejected at the auth step (HTTP 401) with one canonical *rejection* receipt for audit.

6 Receipt semantics

[normative] v0.1 keeps the two-tier classification of RFC-006-B v0.6 and adds the precise inter-receipt linkage INV-4 mandates:

Tier	For transitions that are	Used by
Canonical	`mint_api_key`, `revoke`, `openai.chat_completions` (auth entry per request)	receipt-chain audit; `prev_receipt_hash` chain; persistent
Operational	`<agent>.chat` dispatch; per-frame observability	lossy ring buffer; aggregable rates surfaced through `observe.health`

6.1 Cross-tier linkage

A canonical OpenAI auth receipt and the operational dispatch receipts it triggered are linked by:

- dispatch receipt's `causal_context.Scalar` = canonical auth receipt's invocation_id;
- canonical auth receipt's `state_transition.dispatch_root_invocation_id` = the root operational invocation id (not the receipt id; the invocation id pins the chain head).

A receipt walker traversing from any operational dispatch receipt can therefore reach the canonical auth receipt in one hop (`causal_context`) and recover the API-key URA, the body hash, and the usage numbers.

6.2 What receipts witness, in this reference instance

[normative]

- **mint_api_key receipt** witnesses "user U created API key resource R at time t, token hash H, label L, expiry E".
- **auth receipt (per request)** witnesses "a request bearing key R for model M with body hash H_b and usage $\{i, o, t\}$ entered the system at t', forwarded into dispatch root invocation I_{root}".
- **dispatch receipts (per call)** witness the agent's tool calls + sub-invocations; each cites the auth receipt; together they reconstruct what `<agent>.chat` actually did under this key.
- **revoke receipt** witnesses "key R revoked at t'', reason ρ".

A consumer can therefore answer "how many requests did key R make and what did they cost?" from canonical receipts alone, and "what tools did the agent invoke under each request?" by walking down into operational receipts.

7 URL = invocation, in this reference instance

7.1 Endpoint surface

[normative] The Hub binds three endpoints under the realm's HTTPS listener:

```
GET   /v1/models
POST  /v1/chat/completions
POST  /v1/embeddings                    (deferred; see §10)
```

Only `/v1/chat/completions` is in scope for v0.1 normative behaviour. `/v1/models` is included because the OpenAI-shape clients enumerate it for model discovery; v0.1 returns the realm's chat-base ability registry projected as a list of model names. `/v1/embeddings` is reserved for a future v0.2 once an embedding-base ability primitive lands.

7.2 The transformation

[normative] An HTTP request:

```
POST /v1/chat/completions HTTP/1.1
Authorization: Bearer easynet-sk-<id>
Content-Type: application/json
{
  "model":    "web-builder",
  "messages": [{"role":"user","content":"hi"}],
  "stream":   true
}
```

is translated by `ability/01HUB.openai.chat_completions` into:

```
caller    = hub/01HUB
callee    = hub/01HUB                       (auth ability)
ability   = ability/01HUB.openai.chat_completions
subject   = resource/api_key.<id>           (parsed from Bearer)
args      = { model: "web-builder", messages: [...], ... }
```

which then forwards (per INV-1) into:

```
caller    = hub/01HUB
callee    = agent/web-builder
ability   = ability/web-builder.chat
subject   = user/<resolved-from-api-key>
args      = { prompt: "hi", context: ... }
causal_context = Scalar(<canonical auth receipt id>)
```

The dispatch's frames are filtered (per INV-3) into SSE chunks streamed back as the HTTP response body.

7.3 Why this is not a routing rule

[derived] Per INV-1, the Hub's adapter does not own the dispatch; it forwards. Per INV-3, the bytes the adapter writes to the SSE stream are a deterministic projection of the dispatch frames. Together, INV-1 + INV-3 mean the OpenAI URL form is a projection of the URA form: every OpenAI request has a URA, every OpenAI response equals the URA's invocation result projected through `filter`, and the correspondence is verifiable through the receipt chain.

This is the load-bearing claim of the paradigm extension. The OpenAI API is no longer "a competing protocol" or "an external dependency"; it is a viewing surface onto the invocation graph, the same way HTTP file fetch is.

8 Reference implementation (informative)

This section is non-normative. It witnesses that the v0.1 specification is realisable on current EasyNet-Cli infrastructure. Conforming implementations are not required to follow these specific choices.

8.1 New abilities (Hub + keyring)

Three abilities to register:

```
src/runtime/agents/openai_compat/
  mod.rs              ← register `01HUB.openai.chat_completions` +
                        `01HUB.openai.list_models`
  filter.rs           ← stream-json -> OpenAI SSE projection
                        (deterministic; pure fn)
  auth.rs             ← parse Authorization header; resolve
                        API-key URA via keyring; emit canonical
                        auth receipt
  models.rs           ← /v1/models — project the local ability
                        registry's chat-base entries

src/runtime/keyring/
  api_key_ability.rs  ← <user>.keyring.mint_api_key /
                        <user>.keyring.revoke /
                        <user>.keyring.list_api_keys
  api_key_storage.rs  ← persist to ~/.easynet/keyring/api_keys.toml
                        (reuse RFC-002 vault primitives)

src/runtime/hub/
  openai_listener.rs  ← extend the Hub axum router with
                        /v1/{chat/completions,models}; route to the
                        Hub abilities above
```

8.2 The filter, sketched

[normative] The filter consumes stream-json events and emits OpenAI SSE chunks. Pseudocode:

```
async fn filter(stream: StreamJsonRx, sink: SseSink) {
    let mut completion_id = format!("chatcmpl-{}", random_token());
    let model = current_model_name();
    let created = current_request_time_pin();   // pinned at request
                                                // start; constant
                                                // for the whole stream
                                                // so INV-3 holds.

    sink.send_first_chunk(completion_id, model, created, "assistant");

    while let Some(frame) = stream.recv().await {
        match frame {
            FrameType::AssistantText(text) => {
                sink.send_delta_chunk(completion_id, model, created, text);
            }
            FrameType::AssistantToolUse(tool) if client_supports_tools => {
                sink.send_tool_call_chunk(completion_id, model, created, tool);
            }
            FrameType::AssistantThinking(t) if client_opted_in_reasoning => {
                sink.send_reasoning_chunk(completion_id, model, created, t);
            }
            FrameType::Result { stop_reason } => {
                sink.send_finish_chunk(completion_id, model, created, stop_reason);
                sink.send_done_sentinel();
                return;
            }
            _ => { /* drop: handshake, tool_result, etc. */ }
        }
    }
}
```

8.3 The MVP demo

[normative] The minimum existence proof is one Cursor request that hits the Hub, authenticates against an EasyNet-issued API key, forwards to `web-builder.chat`, and returns the agent's real reply with byte-equivalent OpenAI wire framing. This discharges all four invariants simultaneously: Adapter Purity (the Hub did not invent the reply), Capability-URI Key (the Bearer matched a `resource/api_key.<id>` URA), Filter Determinism (a second Cursor request with identical body returned identical SSE), Auth Receipt Trail (the receipt store shows one canonical auth receipt per request, dispatch receipts citing it).

8.4 Test stratification

[normative] Three matrices, each layered three deep, mirroring RFC-006-B v0.6 §6:

- **Matrix A (unit/integ, in-process)**: filter determinism (same input frames → same SSE bytes), key minting + revocation transitions, malformed-Authorization rejection paths.
- **Matrix B (CLI surface, out-of-process)**: `easynet api-key {create,list,revoke}` subcommands.
- **Matrix C (Docker e2e, real container, real OpenAI client)**: cursor + openai-python

smoke runs against a containerised Hub; SSE round-trip; tool-use rendering; revocation mid-stream.

9 Beyond MVP — post-v0.1 work, in shape only

[deferred]

- **Embeddings** (`POST /v1/embeddings`). Requires an embedding-base ability primitive (`<agent>.embed` or similar). Same paradigm: the response is a deterministic projection of the agent's embedding output, the API key auth is identical.
- **Anthropic-shape adapter** (`POST /v1/messages`). A second filter, same dispatch source. Models the same paradigm against a second external protocol.
- **Gemini-shape adapter** (`POST /v1beta/models/<model>:streamGenerateContent`). A third filter.
- **Scoped API keys**: keys whose admission scope is narrower than "acts as user U". E.g., key may invoke only models in a specific ability namespace, or only at rates below a threshold. Scope is stored on the key resource at mint time; admission consults it.
- **Usage / billing receipts**: enrich the auth receipt's `state_transition.usage` with cost attribution per ability invocation, so an audit can answer "what did this user cost the cluster across all chat-base abilities last month?".
- **Tool-call surface**: explicit OpenAI-shape tool registration so a client can pass `tools=[...]` and have the agent's tool_use frames map onto the client's tool descriptors. v0.1 either drops tool_use frames or summarises them as text; the proper mapping is a v0.2 item.
- **Conversation continuation across requests**. The OpenAI protocol is stateless (every request carries the full `messages` history); EasyNet's chat ability is too. A client that wants "server-side history" must introduce a separate resource (e.g. `resource/conversation.<id>`) and reference it explicitly — RFC-006-D's territory.

Each is purely additive. None changes the four invariants of v0.1; they instantiate the paradigm in new ways or layer additional resources on top.

10 Open decisions

Open decision

API-key bearer prefix. v0.1 specifies `Bearer easynet-sk-<id>` as the user-visible token shape, with `<id>` a 256-bit unguessable hex/base32 token. The `easynet-sk-` prefix is convention only — it lets a secret-scanner (e.g., GitHub's leaked-credential detector) recognise EasyNet keys distinctly from OpenAI's `sk-proj-` or Anthropic's `sk-ant-`. The exact prefix string is non-normative.

Open decision

Model name resolution. v0.1 resolves `model = "web-builder"` to ability `web-builder.chat` by suffix-appending `.chat`. A future revision may permit explicit URAs (`model = "easynet:///r/easynet.run/ability/web-builder.chat"`) for disambiguation across realms, or short aliases stored on the hub. Both fit the paradigm; v0.1 picks the simplest form.

Open decision

Tool-use rendering. Mapping agent tool_use frames to OpenAI `tool_calls` requires a tool descriptor schema the client agreed to up-front (via the `tools=[]` request field). v0.1's normative position is: *drop* tool_use frames from the SSE projection unless the request explicitly listed compatible tools, and *always* record them in the operational receipt chain. v0.2 specifies the descriptor handshake.

Open decision

Revocation propagation latency. INV-2 requires revocation to take effect "without restarting the daemon"; INV-4 requires mid-stream revocation to surface as a `[DONE]` after `finish_reason = "cancelled"`. The implementation needs a watch on the keyring's revocation stream; the latency budget is post-MVP (v0.2).

11 Coverage check

Rule	Discharged by	Where
URA v4.1.5 §A.URA-7 (3-segment URA)	`resource/api_key.<id>` parses to three URA segments	§4
URA v4.1.5 §9 (callee ∈ hub/device/agent)	callee on auth = `hub/01HUB`; callee on dispatch = `agent/<resolved>`	§7
ONTOLOGY §3.2.1 (ability owner ∈ user/agent/hub/resource)	`01HUB.openai.chat_completions` (hub-rooted), `<user>.keyring.mint_api_key` (user-rooted)	§5
AXIOM (every invocation produces a receipt)	two-tier classification + cross-tier linkage	§6
RFC-006-B v0.6 INV-1 (Adapter Purity)	inherited verbatim as RFC-006-C INV-1	§4
RFC-006-B v0.6 INV-3 (Deterministic Projection)	inherited verbatim as RFC-006-C INV-3	§4
RFC-002 keyring vault (capability identity)	API keys minted by keyring abilities; addressed as resource URAs	§4
forward_invoke endgame contract	adapter forwards via standard `forward_invoke`	§7

Closing. Every chat-base ability is, after a deterministic stream filter, an OpenAI streaming completion. Every API key is, by construction, a v4.1.5-conformant `resource/api_key.<id>` URA capability. The OpenAI endpoint is one ability call from a Hub-rooted adapter into a chat-base agent ability, and the wire bytes the client receives are a verifiable projection of the dispatch frames the agent emits. EasyNet does not host LLM endpoints; it lets cursor / continue / openai-python view the invocation graph through the glass cursor / continue / openai-python already speak.

The OpenAI protocol is not a competitor and not a dependency. It is a viewing surface — one of many — onto agents whose work, with or without the surface, is recorded in EasyNet's invocation graph.